Cryptography sheet # 5

G. Averkov

May 15, 2026

In this sheet, we play with another variation of Caesar's cipher, try to understand if the fast exponentiation is actually optimal and do computations to get a feel of the Chinese Remainder Theorem.

1 A non-local variation of Caesar

The problem with all the variations of Caesar's cipher that we've mentioned so far is that there are local in the following sense. The i-th symbol y_i in the cipher text only depends on the i-th symbol x_i of the plain text. This is of course a great weakness when you work over such a small alphabet as the Latin one. For example, if you figured out, for some values of i, what x_i must have been, you can just consider a system of equations and try to get as much information as you can about the key. Say, if you are a cipher breaker and your guess is that the word "THE" occurred in positions i = 100, 101, 102, meaning that the encryption process transformed the letters like this: 'T' $\mapsto y_{100}$, 'H' $\mapsto y_{101}$, 'E' $\mapsto y_{102}$. If $y_i = \mathrm{F}_k(x_i)$, where $\mathrm{F}_k : \mathbb{Z}_{26} \to \mathbb{Z}_{26}$ is the function that reshuffles the alphabet, then you can try solving the system of equations:

$$y_{100} = \mathrm{F}_k(\text{'T'}), \quad y_{101} = \mathrm{F}_k(\text{'H'}), \quad y_{102} = \mathrm{F}_k(\text{'E'})$$

for the unknown key k. Things get more complicated when the encryption is non-local. Let's consider the non-local encryption by the function $\mathrm{f} : \mathbb{Z}_{26}^n \to \mathbb{Z}_{26}^n$ that transforms an n-symbol plain text $x = (x_1, \ldots, x_n)$ to the cipher text $y = (y_1, \ldots, y_n)$ using $y_i = a(x_1 + \cdots + x_i) + b$. The encryption key here is the pair of the values $a, b \in \mathbb{Z}_{26}$.

(a) Analyze for which choices of (a, b) this function is bijective and for which not.

(b) Determine, in the cases of bijectivity, the inverse function of f, which would do the decryption.

Now, let us turn to a more concrete situation. Assume that the above f is bijective and assume, as in the example before, that you know that the plain text contained the word THE in the positions $i = 100, 101, 102$. This means you know that x_{100} = 'T', x_{101} = 'H', x_{102} = 'E'. You also know the whole cipher text y, which you intercepted. Show that

(c) you can use the above information to determine a, but

(d) this information alone will not allow you to determine b.

This exercise demonstrates that with a non-local encryption it is harder to get the whole key from a partial information on the plain text x.

2 Is fast exponentiation optimal?

We have seen fast exponentiation allows calculates the power x^e for $e \in \mathbb{N}$ using the number of multiplications which is (roughly) proportional to the bit size of the exponent e. But can we do better? That is, does there exist a method of computing x^e that would take even smaller number of multiplications? Probably, one should clarify what the word "method" means. Think about the method as the process of generating different powers of x and then combining them to obtain new powers until we get the desired power x^e. Say, if we have powers x^a and x^b computed, then one additional multiplication would give us $x^{a+b} = x^a \cdot x^b$. Similarly, if some x^a is already computed, one additional multiplication will give as $x^{2a} = x^a \cdot x^a$. From this perspective, the process of computing, say x^{10}, is this:

$$\begin{aligned} x^2 &= x \cdot x \\ x^4 &= x^2 \cdot x^2 \\ x^5 &= x^4 \cdot x \\ x^{10} &= x^5 \cdot x^5 \end{aligned}$$

Show that if $e \geq 2^k$, where k is a non-negative integer, then x^e cannot be computed with less than k multiplications. This shows that the fast exponentiation is an optimal method in terms of the number of multiplications.

Hint: you could use induction on k; verify that the assertion is true for $k = 1$, then deduce that whenever the assertion is true for any given non-negative integer k, it is also true for the next value (that is, it is true for $k + 1$ in place of k).

3 EEA meets Chinese Remainder Theorem

In terms of computations, Chinese Remainder Theorem is backed by the Extended Euclidean Algorithm (EEA). Let $z \in \mathbb{Z}$ such that $[z]_{101} = [20]_{101}$ and $[z]_{103} = [15]_{103}$. What is $[z]_{10403}$?

Hint: the output of EEA executed on $(101, 103)$ is helpful. Use them to give an answer.

Hint: you will obtain two integers, say, s and t such that $101s + 103t = 1$. There is something special about $101s$ and $103t$. Try to take them modulo 101 and modulo 103. What cosets will you get? How can this be useful?

4 Solving quadratic equations in modular arithmetic

One of the favorite occupations in a school math class was solving quadratic equations. You would consider an equation like $x^2 - 5x + 6 = 0$ and solve it over reals. This particular one has two solutions $x = 2$ and $x = 3$. Memorizing a formula for solutions of quadratic equation is a kind of math-school religion. You might have two solutions or one solution or even no solution. So, two solutions are the maximum possible number of solutions for a quadratic equation, at least if you do it over reals or over complex numbers. But what about rings $\mathbb{Z}_n$? Try solving the equation $x^2 + x + 1 = 0$ over $\mathbb{Z}_{217}$ or at least to determine the number of solutions.

Hint: Since $\mathbb{Z}_{217}$ is finite, you can enumerate all possible x, but this is not very conceptual and not particularly useful in the cases of $\mathbb{Z}_n$ with a large n. Note that $217 = 7 \cdot 31$. What theorem are you thinking about?

Remark: Why is such kind of exercises relevant for cryptography? Breaking a cipher is solving a complicated equation $\mathrm{f}(x) = y$, where y is the known cipher text and x is the unknown plain

text. We have already broken ciphers based on linear functions by solving linear equations (or systems of linear equations). Solving quadratic equations is a reasonable next step.

5 Assumption of the Chinese Remainder Theorem

Consider integers $n, n_1, n_2 \geq 2$ such that $n = n_1 n_2$ and let $g \coloneqq \gcd(n_1, n_2)$. When $g = 1$, we have shown that the unitary homomorphism $\mathrm{f} : Z_n \to \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2}$ given by $\mathrm{f}([z]_n) = ([z]_{n_1}, [z]_{n_2})$ is bijective (or another words, f is a unitary isomorphism of rings). Show that, when $g > 1$, the homomorphism f is neither injective nor surjective. Just to visualize, what happens when $g > 1$, you could try making table for concrete choices of n_1 and n_2 (say, $n_1 = 2$ and $n_2 = 6$) with rows indexed by $\mathbb{Z}_{n_1}$, columns indexed by $\mathbb{Z}_{n_2}$ and the cell indexed by $[z]_{n1}$ and $[z]_{n_2}$ occupied with $[z]_n$.